



Coinigo Crypto Payment Gateway

API Documentations V1.6.0

March 2026

Comprehensive Integration guide for using CoiniGo as a Payment gateway

1- Introduction

Coinigo is a B2B cryptocurrency payment gateway designed for platforms that manage user balances, trading accounts, or credit accounts.

Coinigo enables two core capabilities:

- **Pay-In (Deposit):** Allows users to deposit cryptocurrency into the merchant's platform.
- **Pay-Out (Withdrawal):** Allows the merchant to send cryptocurrency from its Coinigo wallet to an external blockchain address on behalf of a user.

For Pay-In:

- Coinigo generates deposit addresses (via UI or API).
- When funds are received on-chain, Coinigo sends a webhook notification to the merchant.
- The merchant is responsible for validating the webhook and crediting the correct user account.

For Pay-Out:

- The merchant submits a withdrawal request via API.
- Coinigo validates the request and broadcasts the transaction to the blockchain.
- Status updates are sent via webhook until the transaction reaches a final state.

Coinigo also provides a dashboard where merchants can:

- Monitor wallet balances
- Track Pay-In and Pay-Out transactions
- Review transaction details and operational history

Coinigo acts as the crypto infrastructure layer. Final balance crediting, user accounting, and business logic remain the merchant's responsibility.

2- How to use Coinigo

2-1- Obtain a Merchant Account

To set up a merchant account, businesses must first complete the legal and contractual onboarding process with the Coinigo team. Once finalized, please send the following details to **gw@Coinigo.com** to initiate merchant account and terminal setup:

Required Information:

- Merchant Account Owner's **Email Address**
- Merchant Account Owner's **Full Name**
- **Contact Number**
- Merchant's **Public RSA Key** (for encryption)
- **Webhook Pay-in URL**
- **Webhook Pay-out URL**
- **Return-to-Merchant URL**

After receiving this information, the Coinigo team will:

- ☒ Create your merchant account
- ☒ Configure your terminal
- ☒ Provide the following credentials via email.

Issued Security Credentials – For API & Webhook Integration

These credentials are provided to securely connect and communicate with the Coinigo platform:

Credential	Description
Client ID	Public identifier for your system when calling Coinigo APIs.
Client Secret	Used to authenticate and obtain API access tokens
Webhook Secret – Pay-in	Used to verify incoming webhook events for deposits (Pay-ins).
Webhook Secret – Pay-out	Used to verify incoming webhook events for withdrawals (Pay-outs).
Coinigo Public RSA Key	Used to encrypt API payloads sent to Coinigo.
Unique Digest Secret Key	Used to generate X-Payload-Digest for request integrity

These credentials must be securely stored and never exposed to frontend systems.

2-2- Pay-In Integration (Deposit)

Coinigo supports two methods for Pay-In integration:

- **Coinigo UI (Recommended)**
- **Coinigo API**

The correct choice depends on your technical requirements and internal business logic.

Option A – Coinigo UI (Recommended for Most Merchants)

This is the fastest and simplest Pay-In integration method.

Flow:

1. Merchant requests an access token.
2. Merchant redirects the user to the Coinigo Pay-In UI.
3. Coinigo handles:
 - Asset selection
 - Network selection
 - Deposit address generation
4. When funds are received, Coinigo sends a Pay-In webhook callback.
5. Merchant validates the webhook call and credits the user.

Example (Payment-Based Mode):

- User clicks **“Deposit to Account A”** in the merchant platform.
- Merchant creates a new deposit request internally and generates `paymentTrackingId = PT_12345`. (Optional)
- Merchant requests the UI token including `paymentTrackingId = PT_12345` and redirects the user to CoiniGo UI.
- User sends funds to the generated address.
- CoiniGo sends a Pay-In webhook containing `paymentTrackingId = PT_12345`.
- Merchant validates the webhook and credits **that specific deposit request** (Account A).

Key characteristics:

- No need to build deposit UI
- No need to manage address generation logic
- Reduced risk of wrong network or memo errors
- Minimal development effort

This method is strongly recommended unless full backend automation is required.

Pay-In Operating Models

In UI method, Merchants must choose one Pay-In operating model.

The decision depends on whether you need to register a deposit intent before the user sends funds.

1. Direct Deposit Mode

Use this mode if:

- Users can deposit at any time.
- You do not create a deposit request before funds are sent.
- You credit the user only after receiving a successful Pay-In webhook.
- In Direct Deposit Mode, PaymentTrackingId is not required and may be null in the token request body and the webhooks call.

Flow:

- Deposit address is generated.
- User transfers crypto.
- Coinigo sends webhook with status (Accepted and Completed).
- Merchant validates and credits user balance.

Payment-Based Mode

Use this mode if:

- You must register a deposit request (intent) before the user transfers funds.
- Each deposit must be linked to a specific transaction.
- You use PaymentTrackingId for mapping deposits to predefined requests.

- paymentTrackingId is **required** in Payment-Based mode.
- paymentTrackingId is **optional/null** in Direct Deposit mode.
- **PaymentTrackingId** is created by the merchant per deposit intent.
- In Payment-Based Mode, CoiniGo uses PaymentTrackingId for mapping and returns it in Pay-In webhook.

Flow:

- Merchant creates a deposit intent internally.
- Merchant generates or associates a PaymentTrackingId.
- User transfers funds.
- Coinigo maps the deposit to the correct PaymentTrackingId.
- Merchant credits the corresponding transaction after webhook validation.

Payment-Based mode is suitable for platforms that require strict deposit tracking and intent-based accounting.

PaymentTrackingId Mapping Algorithms

When using the Payment-Based flow, CoiniGo supports three mapping algorithms:

- **FIFO (First In, First Out)**
The earliest pending PaymentTrackingId is matched first.
- **LIFO (Last In, First Out)**
The most recent pending PaymentTrackingId is matched first.
- **Restricted ID**
In Restricted ID mode, only one active paymentTrackingId is allowed per customerIdentifier. Any new request will replace the existing pending paymentTrackingId.

Multi-Account Recommendation:

If a user has multiple trading accounts, it is recommended to assign a unique CustomerIdentifier per account.

This ensures deposit addresses and PaymentTrackingIds are isolated per account and prevents cross-account crediting.

The final mapping logic should be aligned with the merchant's internal account architecture.

Option B – API-Based Pay-In

Use API integration when:

- You want full control over the deposit UI and workflow.
- You want to generate deposit addresses programmatically.

- You need to integrate Pay-In into an existing backend automation pipeline.

With API-based Pay-In, the merchant must:

- Authenticate using Sign-In API.
- Generate deposit addresses via API.
- Implement their own deposit UI.
- Process Pay-In webhooks and perform validation before crediting users

2-3- Pay-Out Integration (Withdrawal)

Pay-Out is always API-based. There is no UI-only Pay-Out mode.

Pay-Out flow:

- Merchant authenticates using the Sign-In API.
- Merchant submits a Pay-Out request:
- Coinigo validates the request.
- Coinigo broadcasts the transaction to the blockchain.
- Coinigo sends webhook updates until the transaction reaches a final state (Completed or Failed).

Important:

- Merchant must monitor webhook updates.
- Merchant must update internal withdrawal status based on webhook status.
- Final withdrawal completion must not rely solely on the initial API response. Merchants must wait for a final webhook status (**Completed** or **Failed**) before marking the withdrawal as finalized in their system.

2-4- Managing Wallet Balance

All Pay-In deposits are credited to the merchant's Coinigo wallet. All Pay-Out withdrawals are deducted from the merchant's Coinigo wallet.

Merchants can manage and monitor wallets through the Coinigo dashboard:

- View available and total balances
- Review Pay-In and Pay-Out history
- Track transaction details

Coinigo manages blockchain interaction and custody at the terminal level.

Merchants remain responsible for internal user balance accounting and reconciliation.

2.5. Secure Communication Protocol & Mandatory RSA Encryption

This section defines the security model used for CoiniGo API integrations.

CoiniGo uses:

- HTTPS (TLS) for transport security
- HMAC SHA1-based X-Payload-Digest for request integrity
- RSA encryption for secured business payload confidentiality

Different endpoints have different security requirements. These must be followed exactly.

2.5.1. Key Exchange

Before integrating secured business APIs:

- CoiniGo provides its Public RSA Key to the Merchant.
- The Merchant must provide its Public RSA Key to CoiniGo.
- CoiniGo securely shares a Digest Secret Key.

Important:

- The Digest Secret Key is used only to generate X-Payload-Digest.
- Webhook Secret Keys are used exclusively for webhook verification. CoiniGo uses the Webhook Secret Key to generate an X-Payload-Digest, which is included in the webhook request header. Merchants may optionally regenerate the X-Payload-Digest after decrypting the request body and compare it with the value sent by CoiniGo. This

additional validation step enhances security, while RSA encryption ensures payload confidentiality.

- Private keys and secret keys must never be exposed in frontend systems.

2.5.2 General Security Rules

1. Secured business APIs require both RSA encryption and X-Payload-Digest.
2. Authentication and UI bootstrap endpoints use plaintext JSON.
3. When digest is required, it must be generated over the original plaintext JSON body before encryption.

Each endpoint section explicitly states its requirement as compared in the table below:

Category	RSA Encryption	X-Payload-Digest	Payload Format	Key Notes
UI Token Endpoint	Not Required	Required	Plaintext JSON	Must send direct JSON fields (clientId, clientSecretKey, customerIdentifier, returnUrl). Do NOT wrap inside dataEncrypted.
Signin Endpoint	Not Required	Required	Plaintext JSON	Used to obtain JWT for secured APIs.
Secured Business APIs	Required	Required	Encrypted JSON inside dataEncrypted	Digest must be generated over plaintext JSON before RSA encryption. Raw JSON must NOT be sent.

Additional Clarifications

- If **RSA = Required**, payload must be encrypted and sent inside dataEncrypted.
- If **RSA = Not Required**, payload must be sent as plaintext JSON.
- **Digest** must be generated using HMAC-SHA1 over the original plaintext JSON body.

2.5.5 Execution order for APIs with both RSA encryption and X- Payload digest

Examples:

- Generate Address
- Pay-Out Requests
- Wallet Information
- Transaction Queries

For all secured business APIs:

- RSA Encryption: Required
- X-Payload-Digest: Required
- Payload must be sent inside dataEncrypted

Required Execution Order

Step 1 — Prepare plaintext JSON body

Step 2 — Generate X-Payload-Digest using HMAC-SHA1 over plaintext body

Step 3 — Add X-Payload-Digest header

Step 4 — Encrypt plaintext JSON using CoiniGo Public RSA Key

Step 5 — Send encrypted payload inside:

```
{  
  "dataEncrypted": "RSA_ENCRYPTED_PAYLOAD"  
}
```

2.5.6 Validation Flow on CoiniGo Side

For secured business APIs:

1. CoiniGo decrypts dataEncrypted.
2. CoiniGo validates X-Payload-Digest against the decrypted plaintext payload.
3. Business validation is performed.

If validation fails, the request is rejected with errors such as:

- bad_request

- `invalid_digest`

2.5.7 Webhook Handling by Merchant

For encrypted callbacks:

- Merchant must decrypt the callback using its Private RSA Key.
- Coinigo uses the Webhook Secret Key to generate an X-Payload-Digest, which is included in the webhook request header.
- Merchants may optionally regenerate the X-Payload-Digest after decrypting the request body and compare it with the value sent by Coinigo.
- Business logic must rely only on successfully decrypted data.
- HTTP 200 status alone does not indicate business success.

2.5.9 Generating the X-Payload-Digest

The Merchant is responsible for generating the X-Payload-Digest value, following the implementation guidance provided by Coinigo . Refer to the provided code samples for exact structure and logic.

```
public static class RequestBodySecurityHelper
{
    public static string GenerateDigest(string secretKey, string requestBody)
    {
        var encoding = new ASCIIEncoding();

        var messageBytes = encoding.GetBytes(requestBody);
        var keyBytes = encoding.GetBytes(secretKey);

        using var hmac = new HMACSHA1(keyBytes);
        var hashedValue = hmac.ComputeHash(messageBytes);
        return Convert.ToBase64String(hashedValue);
    }
}
```

X-Payload-Digest is required for UI Token, Authentication, and all Secured Business APIs.

3- Get Started with the Integration:

CoiniGo supports two integration approaches:

1. **UI-Based Pay-In (Recommended for faster onboarding)**
2. **API-Based Integration (Full control – required for Pay-Out)**

Choose the approach that matches your technical and business requirements.

3-1- Using CoiniGo UI for Pay-In (Deposit)

The UI-based integration allows merchants to handle deposits through CoiniGo's hosted interface. This is the fastest and simplest integration method.

The merchant only needs to:

- Implement webhooks
- Request a UI access token
- Redirect users to CoiniGo

CoiniGo handles address generation, network selection, and user interaction.

Key Characteristics

- No need to build deposit UI
- No need to manage blockchain address logic
- Reduced wrong-network risk
- Minimal backend implementation

Step 1: Implement Webhooks (Mandatory)

Before enabling deposits, you must implement webhook endpoints to receive:

- Pay-In notifications
- Pay-Out notifications

Your webhook endpoint must:

- Accept HTTP POST requests
- Return HTTP 200 OK
- Implement idempotency using RequestId and/or TransactionHash

Refer to the **Webhook Notification Guide** in the appendix for payload structure and validation rules.

Without webhook handling, you cannot safely credit user balances.

Step 2. Obtain UI Access Token

To redirect users to CoiniGo UI, you must first request an access token.

API Endpoint:

POST /ipg/b2p/crypto/token

Request Field Descriptions:

- **clientId**: Your unique **Client ID**, provided by the Coinigo team.
- **clientSecretKey**: Your **Client Secret**, also provided during onboarding.
- **customerIdentifier**: Merchant-defined identifier used to uniquely map deposit addresses and associate transactions with the intended internal entity (user, account, or any merchant-defined reference).
- **returnUrl**: The URL where you want the user to be redirected after completing the deposit address generation process.
- **traceNumber**: Reserved for future business enhancements.
- **customerUserAgent**: The browser user agent of the end user, required for security purposes.
- **customerIp**: The IP address of the user, also required for security validation.

- **paymentTrackingId**: Merchant-generated identifier for a deposit intent (payment request). **Required** in **Payment-Based Mode**. **Optional / null** in **Direct Deposit Mode**. Returned in **Pay-In webhook** as PaymentTrackerId (nullable)
- **paymentExpirationTime**: Defines the validity period of the requested paymentTrackerId. If a transaction is received after the specified paymentExpirationTime has expired, the paymentTrackerId will be returned as null in the callback. If paymentExpirationTime is provided with a value earlier than the current server time, the request will be rejected with the following validation error:
"PaymentExpirationTime must be greater than the current time."

```
{
  "clientId": "CLIENT_123",
  "clientSecretKey": "SECRET_456",
  "customerIdentifier": "USER_789",
  "returnUrl": "https://merchant.com/callback",
  "customerUserAgent": "Mozilla/5.0",
  "customerIp": "192.168.1.10",
  "paymentTracker": {
    "paymentTrackingId": "PAY_2025_0001",
    "paymentExpirationTime": "2025-01-01T12:00:00Z"
  }
}
```

Digest must be generated over this exact JSON body before sending.

Step 3: Redirect Users to Coinigo UI for Deposit Address Generation

Once you obtain the access **token**, you can redirect your users to the following URL:

<https://gw.coinigo.com/?token=TOKEN>

The user will:

- Select asset and network
- View deposit address and memo (if required)
- Scan QR code or copy address
- Send funds from their wallet

When funds are received and confirmed, CoiniGo will send a Pay-In webhook.

The merchant must:

- Validate webhook
- Ensure Status = Completed
- Validate asset, network, contract address
- Prevent duplicate crediting
- Credit user balance internally

3-2- Leveraging Coinigo 's APIs

API-based integration provides full control over:

- Address generation
- Deposit flow
- Withdrawal (Pay-Out)
- Wallet management
- Transaction querying

This approach is required for Pay-Out functionality.

Base EndPoint

Base URL for accessing the Coinigo API:

<https://gw.coinigo.com/api/>

3-2-1-Sign in API

To interact with Coinigo 's secured APIs, you must first obtain an **access token** via authentication. This token must be included in the header of **every subsequent API call** that requires authorization.

Endpoint

POST /ipg/sign-in

API Body Request Sample:

json

```
{
  "clientId": "your-client-id",
  "clientSecret": "your-client-secret"
}
```

API Response Sample:

```
{
  "requestId": "d1dce3ea2f0b4f26a5586c374294cd75",
  "apiVersion": null,
  "data": {
    "accessToken":
"eyJhbGciOiJSUzI1NiIsImtpZCI6IjJDRDUwNUU1MERGOTkyQjgyODMyOTUxMzgy
liwidHlwIjoieXQrand0In0.eyJpc3MiOiJodHRwczovL2NnLWF1dGgtYXBpLm50LXN0YWdpb
mcuZGV2IiwibmJmljoxNzU0NDkzMTC3LCJpYXQiOiJ3NTQ0OTMxNzcslmV4cCI6MTc1Njly
MTE3NywiYXVkljoiaHR0cHM6Ly9jZy1hdXRoLWFWaS5udC1zdGFnaW5nLmRldi9yZXNvdX
JjZXMiLCJzY29wZSI6WyJ0ZXJtaW5hbC1hcGktYWVWZmVudF9pZCI6IHRvcENo
YW5nZV9UY1N0YWdlXzQiLCJjbGllbnRfVGybyWluYWxfSWRlbnRpZmllciI6IjEwMDAwMCJ
9.hwefBrIVTlx2_mVPxhusiV6GNNM_MEbTpRQ8uHx7Agj60YU70uA4-kbvbrSB3-
3ZYHzvB8icT8MZPp1YPN9Gdxko0ciJea05pqBTPeDuKDqofg5_1BjLyqUZ3LmA-
8FfEeoO4x-
9oK6HGY0StcA95IsJOZy_hBOWcYE78GVbjar8LxQY7b7V02hxuiSEzysDAXHGvciiDBBUuEK
Y0zaach0lqUdFjWtJDyaz6VP34PjhbSfHdU9yUthsCwhNSdmJmfH_FtY1ZWeRwwHQ5I2Rv
sWzD2l3JsEX-ezesFMpo48wBRjzEi08ZhV9JloosnyncOGJqVkQstCgx79oFnG2TA",
    "tokenType": "Bearer",
    "expiresIn": 1728000
  }
}
```

3-2-2- Generate Address

Generates a unique deposit address for a specific network and customer.

API Endpoint: /ipg/cryptos/addresses

API Body Request

```
json
{
  "network": "string",
  "label": "string",
  "customerIdentifier": "string"
}
```

API Request Body Fields

- **network:** The network supported by your terminal (e.g., BTC, ETH, TRX).

- **label** (*optional*): A free-text field to add notes or details about the transaction.
- **customerIdentifier**: A merchant-defined identifier used to uniquely map deposit addresses and associate blockchain transactions with the intended internal entity. This value may represent a user, a specific account, or any internal reference defined by the merchant's business logic. CoiniGo treats this value as an opaque identifier and does not interpret its structure

API Response

```
json
{
  "requestId": "string",
  "data": {
    "address": "string",
    "memo": "string (if applicable)",
    "network": "string"
  }
}
```

Note:

Don't forget to include your **JWT access token** in the request header:

3-2-3- Assets

The **Get Supported Assets** API returns a list of cryptocurrencies that are configured and available for your terminal. These assets can be used for initiating Pay-in and Pay-out operations. The response includes essential details such as the asset's ID, name, symbol, associated blockchain network, decimal precision, and logo URL for UI purposes. This endpoint helps your system dynamically retrieve and display the list of supported assets without hardcoding them.

API Endpoint: /ipg/cryptos/assets

API Body Request

This API does **not** require a request body.

API Response

```
json
{
  "requestId": "string",
  "data": [
    {
      "id": "string",
      "name": "string",

```

```

    "symbol": "string",
    "network": "string",
    "decimals": number,
    "logoUrl": "string"
  }
]
}

```

Response Field Description

- **id**: Unique ID representing the currency (e.g., USDT, BTC).
- **name**: Full name of the asset (e.g., Tether USD).
- **symbol**: Currency symbol (e.g., USDT).
- **network**: Associated blockchain network (e.g., TRON, Ethereum).
- **decimals**: Number of decimal places supported by the currency.
- **logoUrl**: URL for the asset's logo (optional, for UI display).

Note

This API returns the list of **assets configured for your terminal**. It helps you understand which currencies are available for Pay-in and Pay-out operations.

3-2-4- Get Networks

Get network details for a specific currency using its ID.

- **Endpoint**: GET /ipg/cryptos/{currencyId}/networks
- **Request Body**: *None*
- **Response**:

```

json
[
  {
    "networkName": "Ethereum",
    "networkCode": "ETH",
    "addressRegex": "^0x[a-fA-F0-9]{40}$",
    "isDefault": true
  }
]

```

3-2-5- Wallet Info (Retrieve Wallet Information)

Description:

Get a list of available wallets, their balances, and related information.

- **Endpoint:**
GET /ipg/crypto/wallets?currencyCode='currencyCode'&dataEncrypted='encryptData'
- **Request Body:** *None*
- **Response:**

```
json
[
  {
    "walletName": "wallet1",
    "accountNumber": "ACC-123456",
    "currencyName": "Bitcoin",
    "currencyCode": "BTC",
    "availableBalance": 0.5423,
    "totalBalance": 0.6500
  },
  {
    "walletName": "wallet2",
    "accountNumber": "ACC-789012",
    "currencyName": "Ethereum",
    "currencyCode": "ETH",
    "availableBalance": 1.2350,
    "totalBalance": 1.5000
  }
]
```

3-2-6- Pay-In Transactions (Get Pay-In List)

Description:

Retrieve a list of recent deposit transactions.

- **Endpoint:**
GET /ipg/crypto/pay-ins?dataEncrypted='encryptData'
- **Request Body:** *None*
- **Query Parameters (Optional):**
 - requestId
 - transactionHash
 - status
 - transactionDate
 - paging.size
 - paging.index
- **Response:**

```
json
```

```
[
  {
```

```

    "walletName": "Main Wallet",
    "accountNumber": "ACC-10001",
    "currencyName": "Bitcoin",
    "currencyCode": "BTC",
    "availableBalance": 0.8500,
    "totalBalance": 1.0000
  }
]

```

3-2-7- Get Details of a Specific Deposit

Description:

Returns detailed information about a specific pay-in transaction.

- **Endpoint:**
GET /ipg/crypto/pay-ins/details?dataEncrypted='encryptData'
- **Request Body:** *None*
- **Query Parameters (Optional):**
 - requestId (integer)
 - transactionHash (string)
- **Response:**

```

json
{
  "requestId": 123456789,
  "amount": 150.75,
  "feeAmount": 0.25,
  "currencyCode": "USDT",
  "currencyName": "Tether",
  "networkName": "Ethereum Mainnet",
  "networkCode": "ETH-MAIN",
  "transactionHash": "0xabc123...",
  "toAddress": "0xReceiverAddress123",
  "fromAddress": "0xSenderAddress456",
  "explorerUrl": "https://etherscan.io/tx/0xabc123...",
  "contractAddress": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
  "status": "Completed",
  "transactionDate": "2025-08-06T15:23:45.123Z"
}

```

3-2-8- Pay-out Request

Description:

Creates a request to send funds to an external wallet.

- **Endpoint:** POST /ipg/crypto/pay-outs/requests

- **Request Body:**

```
json
{
  "referenceNumber": "REF123456789",
  "network": "ETH",
  "address": "0xAbcdef1234567890abcdef1234567890abcdef12",
  "amount": 250.75,
  "currencyCode": "USDT",
  "contractAddress": "0xdAC17F958D2ee523a2206206994597C13D831ec7",
  "customerIdentifier": "Cust-0012345",
  "memo": "Optional memo text",
  "note": "This payout is for invoice #1234"
}
```

- **Response:**

```
json
{
  "dataEncrypted": "****"
}
```

ContractAddress Field Clarification

The contractAddress field represents the smart contract address of a token on the selected blockchain network.

For token transfers (e.g., USDT on TRC20, ERC20, BEP20), contractAddress is required. For native coins (e.g., BTC), contractAddress must be null.

The contractAddress ensures correct asset identification and prevents wrong-network or wrong-token transfers.

CoiniGo validates the following before broadcasting a Pay-Out transaction:

- Network
- referenceNumber
- CurrencyCode
- ContractAddress
- Address format

If any mismatch is detected, the request will be rejected before broadcast.

3-2-9- Pay-out List

Get List of Payouts

Description:

Returns a list of payout transactions initiated by your terminal.

- **Endpoint:** GET /ipg/crypto/pay-outs?dataEncrypted='encryptData'
- **Request Body:** *None*
- **Query Parameters (Optional):**
 - requestId
 - transactionHash
 - status
 - transactionDate
 - paging.size
 - paging.index
- **Response:**

json

```
{
  "requestId": 987654321,
  "referenceNumber": "REF123456789",
  "networkName": "Ethereum Mainnet",
  "networkCode": "ETH-MAIN",
  "amount": 250.75,
  "address": "0xAbcdef1234567890abcdef1234567890abcdef12",
  "memo": "Optional memo text",
  "status": "Pending",
  "note": "This payout is for invoice #1234"
}
```

Appendix: Webhook Notification Guide

Overview

Coinigo's system automatically sends webhook notifications to your server whenever a cryptocurrency transaction is processed. This section outlines the structure, fields, and behavior of these webhook notifications.

Notification Triggers

Webhook notifications are triggered for the following transaction types:

- **PayIn** – A deposit into your system
- **PayOut** – A withdrawal from your system

Webhook Status Lifecycle

Pay-Out Status Lifecycle

For Pay-Out transactions, CoiniGo sends multiple webhook callbacks as the transaction progresses through its lifecycle.

Possible statuses (in order):

Status	Meaning	Merchant Action
New	Pay-Out request created internally.	No balance update yet.
Accepted	Request validated and queued for processing.	No final action.
Broadcast	Transaction submitted to blockchain network.	Do not mark as completed yet.
Processing	Transaction awaiting sufficient blockchain confirmations.	Wait for final state.
Completed	Transaction confirmed on-chain and finalized.	Mark withdrawal as completed.
Failed	Transaction rejected or failed.	Mark withdrawal as failed

Important Rules

- Webhooks are sent at each stage.
- Merchants must not treat Accepted, Broadcast, or Processing as final.
- Only Completed or Failed represent Final states.
- Idempotency must be enforced using **RequestId** and **TransactionHashID**.

Pay-In Status Lifecycle

For Pay-In transactions, CoiniGo sends webhooks in the following statuses:

Status	Meaning	Merchant Action
Accepted	Deposit detected but not yet fully confirmed.	Do NOT credit user.
Completed	Deposit fully confirmed on-chain.	Validate and credit user balance.

Important Rules

- Accepted means funds are seen but not finalized.
- Only Completed should trigger user balance credit.
- Merchant must validate:
 - CustomerIdentifier
 - CurrencyCode
 - NetworkCode
 - ContractAddress
 - RequestId or TransactionHash (for idempotency)

Webhook Format

- The webhook is sent as an **HTTP POST** request to your configured webhook URL.
- The payload is in **JSON** format and includes comprehensive transaction details.

Fields in the Webhook Notification

Field	Description
Status	Current status of the transaction: New, Accepted, Broadcast, Processing, Completed, or Failed.
TransactionType	Type of transaction: PayIn or PayOut.
DateOnUtc	Timestamp of the transaction in UTC.
RequestId	Unique identifier for the transaction request.
TransactionHash	Blockchain transaction hash (if applicable).
CurrencyCode	Cryptocurrency code (e.g., BTC, ETH).
NetworkName	Name of the blockchain network used.
NetworkCode	Abbreviation of the blockchain network (e.g., BTC, ETH, BSC, TRX, MATIC).
ToAddress	Destination wallet address.
FromAddress	Source wallet address.
ContractAddress	Smart contract address, if applicable.
Amount	Amount of cryptocurrency involved.
FeeAmount	Transaction fee, if applicable.
CustomerIdentifier	he merchant-defined identifier used for mapping (user/account/composite)
PaymentTrackingId	Deposit intent identifier (nullable; populated in Payment-Based mode, may be null in Direct Deposit mode).

Example Scenario

Here you can see two sample notifications for PayIn and PayOut transactions.

Pay-In Example When PaymentTrackingId is populated

```
{
  "PaymentTrackingId": "xxxx",
  "TerminalId": 100000,
  "CustomerIdentifier": "xxxx",
  "Status": "Accepted",
  "TransactionType": "PayIn",
  "DateOnUtc": "xxxx",
  "RequestId": 22000044,
  "TransactionHash": "xxxxxx",
  "CurrencyCode": "xxxx",
  "NetworkName": "xxxxx",
  "NetworkCode": "xx",
  "ToAddress": "xxx",
  "FromAddress": "xx",
  "Amount": 0.0,
```

```
"ContractAddress": "xxxxxx",  
"FeeAmount": null  
}
```

When PaymentTrackingId is empty

```
{  
  "PaymentTrackingId": null,  
  "TerminalId": 100000,  
  "CustomerIdentifier": "xxxx",  
  "Status": "Accepted",  
  "TransactionType": "PayIn",  
  "DateOnUtc": "xxxx",  
  "RequestId": 22000044,  
  "TransactionHash": "xxxxxx",  
  "CurrencyCode": "xxxx",  
  "NetworkName": "xxxxx",  
  "NetworkCode": "xx",  
  "ToAddress": "xxx",  
  "FromAddress": "xx",  
  "Amount": 0.0,  
  "ContractAddress": "xxxxxx",  
  "FeeAmount": null  
}
```

Pay-In Credit Validation Rules (Merchant Responsibility)

Upon receiving a Pay-In webhook, the merchant must validate the following before crediting user balance:

1. Status must be "Completed".
- "Accepted" indicates pending confirmation and must NOT be credited.
2. CustomerIdentifier must match the intended account.
3. CurrencyCode, NetworkCode, and ContractAddress must match the expected asset.
4. ToAddress must be a valid deposit address assigned to the user.
5. RequestId and/or TransactionHash must be used to prevent duplicate crediting (idempotency control).

CoiniGo provides transaction data but final credit logic and validation remain the merchant's responsibility.

Pay-Out webhook Example

```
{
  "ReferenceNumber": "REF123456789",
  "FailReason": null,
  "TerminalId": 100000,
  "CustomerId": "xxxx",
  "Status": "Completed",
  "TransactionType": "PayOut",
  "DateOnUtc": "xxxx",
  "RequestId": 33000055,
  "TransactionHash": "xxxxxx",
  "CurrencyCode": "xxxx",
  "NetworkName": "xxxxx",
  "NetworkCode": "xx",
  "ToAddress": "xxx",
  "FromAddress": "xx",
  "Amount": 0.0,
  "ContractAddress": "xxxxxx",
  "FeeAmount": 0.0
}
```

Important Notes

- Your server must be configured to listen for incoming HTTP POST requests at the webhook URL you provide.
- You must return an HTTP 200 OK response immediately after successfully receiving and validating the webhook.
- The webhook endpoint should not return malformed JSON or unexpected response bodies.
- If your server fails to respond (e.g., timeout, 4xx or 5xx status code), CoiniGo will retry delivery according to the configured retry policy.
- Webhook notifications may be delivered more than once. The merchant must implement idempotency controls using RequestId and/or TransactionHash to prevent duplicate processing.
- If a transaction is successfully executed on the blockchain but an internal error occurs on CoiniGo's side, you will still be notified about the transaction and its final outcome.

This webhook system ensures that you are promptly informed of all transaction events. For any issues or integration assistance, please contact our support team.